

ONIQ MASTER SUPERLOOP — PASS 06: SECURITY AND PRIVACY

0. TARGET — PASS HALTED AS INSTRUCTED

****This project has exactly one database, and it is production.****

- Project ref `bqwttemnnoexadpwifcj`, not paused, Managed by Lovable.
- `.env` binds the app to that same ref (`VITE\_SUPABASE\_PROJECT\_ID`, `VITE\_SUPABASE\_URL`).
- `supabase/` contains only `config.toml`, `functions/`, `migrations/`. There is no branch database and no seeded non-production project.
- The database holds real user data: 20 profiles, 833 messages, 57 conversations, 325 call logs, 4 health profiles, 46 device tokens, 25 CV documents.

Per your own rule, I ****stopped**** rather than proceeding. I did not create USER_A or USER_B, did not attempt a single cross-user read, and did not sign in as anybody. Sections 1 (behavioural half), 2, 3, 5 and 6 are ****BLOCKED — NO NON-PRODUCTION TARGET****, not "passed".

What I did run is metadata and static analysis only: catalogue queries that read schema and policy definitions, HTTP fetches of the already-public JS bundle, and source reads. Zero rows of user data were read.

Proposed target, for your approval

1. Create a second Lovable Cloud project as `oniq-staging`.
2. Replay `supabase/migrations/` into it — the schema is fully migration-defined, so it reproduces exactly, policies included.
3. Deploy the 53 edge functions there with throwaway provider keys (or the provider stubs, since most functions degrade gracefully when a key is unset).
4. Seed USER_A and USER_B through the app's own signup path, so triggers and consent rows fire the way they do for a real user.
5. Point a local build at that project and run Pass 06 in full.

That is a paid second project and a provider-key decision, so it is your call, not mine. Nothing below substitutes for it.

P0 FINDINGS

P0-1 — `profiles` exposes `is\_admin` and `oniq\_pay\_enabled` to every signed-in user

Policy `profiles_select_public_columns`, PERMISSIVE, SELECT, role authenticated, USING (`auth.uid()` IS NOT NULL) — whole row, every row. Any logged-in user can enumerate which accounts are administrators and which have payments enabled.

This is the *fourth* time this finding has come back. Previous fixes have regressed, which means the fix keeps being written as a policy tweak that a later migration overwrites. The durable shape is a column-scoped grant or a `profiles_public` view with the flags absent from the base grant.

You told me not to repair anything in this pass, so I left it live and did not mark it fixed. It is also the one active error-level finding on the project's scanner. **It needs a decision this week, not next pass.**

P0-2 — Ting has no safety guardrails in its system prompt at all

`supabase/functions/ting/index.ts` lines 10-13. The entire system prompt is three sentences of persona: who Ting is, what ONIQ is, answer in the user's language, cite sources. There is **nothing** instructing it to refuse to invent prices, availability, bookings, medical certainty or government approvals; no anti-injection framing; no refusal policy; no PII policy.

Section 5 asked me to confirm the guards hold. Statically, there are no guards to hold. That is a stronger result than a failed red-team run and it needed no production access to establish. Note this ships in a Play listing rated 12+ with a Vitals health surface one tab away.

Cross-user context leakage specifically is **CANNOT_VERIFY** — the function reads only the caller's own request body and bearer token, so there is no obvious cross-user retrieval path in source, but proving it requires two live accounts.

1. RLS — TABLE-BY-TABLE (metadata half only)

Enumerated all 111 public base tables.

| Result | Count |

| --- | --- |
| Tables in public | 111 |
| RLS enabled | **111 (100%)** |
| RLS disabled | **0** |
| FORCE ROW LEVEL SECURITY | 0 |
| Total policies | 200 |
| Tables with RLS on but **zero** policies | 7 |

****No table lacks RLS. There is no P0 of that kind.**** The site's "RLS-enforced data" claim survives the metadata half of the test.

****Anon reachability (Section 1, anon sub-item) — answered without touching data:****
the anon role holds ****zero**** SELECT grants and ****zero**** INSERT/UPDATE/DELETE grants on ****every**** table in public. Anonymous Data API reads are refused at the grant layer before RLS is even consulted. This is the strongest possible answer and it is a catalogue fact, not a probe.

****Zero-policy tables**** (locked, not leaking — but any direct client read fails):
api_budget, copyright_notices, otp_attempts, profile_qr_tokens,
story_dispatch_health, study_chapters_debug, study_papers.
study_papers is the one to look at: 87 real rows that users read, so every path to it must be a SECURITY DEFINER RPC. ****P2.****

Vitals — reported separately, as asked

Highest-sensitivity surface, and it is the best-constructed one in the schema.

Each of health_profiles, health_checkins, cycle_logs carries two stacked policies:

- PERMISSIVE `ALL` — `USING (auth.uid() = user\_id)`, `WITH CHECK (auth.uid() = user\_id)`
- RESTRICTIVE `ALL` — `WITH CHECK (health\_data\_allowed(user\_id))`

Restrictive-plus-permissive is the correct pattern: ownership grants access, the regional guard can only subtract. On the metadata evidence Vitals is sound. The behavioural proof (USER_B reading USER_A's cycle log) is BLOCKED.

Other named tables

- `messages` — SELECT gated on `is\_conversation\_member`; INSERT requires sender = caller AND membership AND (for channels) owner/admin role. ****No DELETE policy at all****, so deletes are refused outright. Well-formed.
- `audit\_log` — SELECT `auth.uid() = user\_id OR is\_admin(auth.uid())`. Own-or-admin.

- ``consent_records`` — read-own, insert-own, no UPDATE or DELETE policy. Append-only by construction. Correct for a consent ledger.
- ``video_jobs``, ``episode_jobs`` — `SELECT `is_admin`(auth.uid())` only. Correct.`

One thing worth a second look

31 policies are attached to role public rather than authenticated. Because anon has no table grants, they cannot currently be exercised anonymously — but they are only safe **because of the grant layer**, not because of their own predicate. A future GRANT SELECT ... TO anon on any of those tables would widen access silently. ****P2, defence-in-depth.****

Positive: ****every**** SECURITY DEFINER function in public pins search_path — 0 exceptions out of the full set. That hardening held.

2. IDOR / DIRECT OBJECT ACCESS — BLOCKED, with one static result

All seven storage buckets are ****private**** (public = false):

chat-media, clips, moments, story-plates, themes, verification-docs, video-gen. A constructed public URL therefore cannot fetch USER_A's upload — that path is closed at the bucket level.

Signed-URL scoping and actual expiry: ****CANNOT_VERIFY**** (needs two live accounts).

Sequential/guessable ID survey: ****CANNOT_VERIFY**** — IDs are uuid throughout the tables I inspected, but proving no endpoint accepts a guessable alternate key needs live requests.

3. AUTHENTICATION AND PRIVILEGE — static result is good, runtime BLOCKED

****Privilege escalation, the part you flagged.**** I read the gates rather than calling them:

- ``src/lib/runway.functions.ts`` — all 7 exported server functions carry `.middleware([requireSupabaseAuth])``, and ``src/lib/runway.server.ts`` re-checks ``rpc('is_admin', { _uid: userId })`` with the service role before doing anything, logging and refusing non-admin callers. Its own header comment states the position correctly: "THE SERVER CHECK IS THE GATE. The admin screen being unlinked is cosmetic."

- `src/lib/episode.functions.ts` — same shape, admin gate first, then kill switch.

So the admin surface is gated at the server, not by a hidden UI. That is the right answer to Section 3. Actual status codes for an ordinary user hitting them:

****CANNOT_VERIFY****.

No-token / malformed / expired / another user's token, and post-logout token reuse: ****BLOCKED****. All are behavioural.

4. SECRET EXPOSURE — CLEAN

Downloaded all 27 shipped JS chunks from <https://onihqhub.com> (732 KB total) and scanned for `sb_secret_`, `service_role`, `SERVICE_ROLE`, `sk-ant-`, Google Alza..., `rzp_live_`, PEM headers, `SUPABASE_SERVICE`.

One apparent hit, in `client-BXrAQIAU.js`. It is the key-format validator: `startsWith("sb_publishable_") || startsWith("sb_secret_")` — a literal in a type check, no value attached. ****Not an exposure.****

The only credential in the bundle is the publishable Supabase key, which is designed to be public. Source-side, the only `import.meta.env` values referenced anywhere in `src` are `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`, `VITE_SUPABASE_PROJECT_ID`. ****No P0 here.****

Debug endpoints: `/app/diag` is referenced in the shipped bundle. It is authenticated (under `_authenticated`) and deliberately unlinked, so it is reachable by URL for signed-in users. ****P3**** — decide whether it should be admin-gated. No `/mcp/*` or `/api/public/*` path appears in the client bundle.

Error-response leakage: ****CANNOT_VERIFY**** — provoking real error responses means malformed requests against production endpoints.

5. AI SECURITY — see P0-2

Everything else in Section 5 (injection, prompt extraction, cross-user leakage, PII, malicious URLs, long/empty/mixed-language input) is ****BLOCKED****. Red-teaming Ting means live calls that spend real provider credit on your metered keys and

write to production tables — both a production write and a money decision.

6. RATE LIMITING — implementation reviewed, thresholds unverified

| Function | Auth | Rate limit |

| --- | --- | --- |

| ting, translate, smart-scout, hotel-scout, story-plot, story-still, story-voice | requireAuth | 10 calls / 60 s, keyed on JWT sub |

| send-push | getUser() → 401 | present |

| estimate-fares | requireAuth | ****none**** |

| send-otp / verify-otp | pre-auth by design | DB-backed throttle via otp_attempts, returns 429 |

****Weakness (P2):**** the limiter is `const rlBuckets = new Map()` — per-isolate, in-memory, resets on cold start. Under Deno's isolate model a caller spread across isolates gets a multiple of 10/min, and a cold-start loop resets the window. On your metered AI keys that is a spend-amplification path, not just an abuse one. Durable limiting belongs in Postgres, next to `api_budget`.

****Note (P3):**** every function above sends `Access-Control-Allow-Origin: *` while accepting Authorization. Not exploitable on its own — browsers won't attach credentials to a wildcard-CORS request — but it means any origin can invoke these with a stolen token.

Repeated-auth-attempt limiting on Supabase Auth itself: ****CANNOT_VERIFY****.

FINDINGS, RANKED

| # | Sev | Finding |

| --- | --- | --- |

| 1 | ****P0**** | profiles leaks `is_admin` / `oniq_pay_enabled` to every authenticated user (4th regression) |

| 2 | ****P0**** | Ting system prompt contains no safety guardrails whatsoever |

| 3 | P2 | `study_papers`: 87 rows, RLS on, zero policies — confirm all reads are DEFINER RPCs |

| 4 | P2 | AI rate limiter is per-isolate in-memory; bypassable, amplifies metered spend |

| 5 | P2 | 31 policies bound to role public; safe only because anon holds no grants |

| 6 | P2 | estimate-fares has auth but no rate limit |

| 7 | P3 | `/app/diag` reachable by URL for any signed-in user |

| 8 | P3 | Wildcard CORS on all credentialed edge functions |

No P1 was established, because every "is the core broken" question in this pass is behavioural and therefore blocked.

COULD NOT VERIFY — AND WHY

Sections 2, 3 (runtime), 5 and 6 (thresholds), plus the behavioural half of Section 1. Single cause: ****no non-production database exists****, and every one of those tests is either a deliberate cross-user read of real user data, a write to production, or real spend on metered provider keys. None of them get a green mark here. Approve a staging target and they all become runnable in one pass.

Nothing was repaired, per your instruction — including the active P0 scanner finding, which I have deliberately left open and unmarked for your decision.